DigiSys final report : CAST-256

| CANO Louis | KAH Frédéric | HIRSCHMILLER Thomas |

Here is the final report of our group for the DigiSys project of implementing a CAST-256 crypto engine on a Zybo Z7 SoC, with GCM mode, and calling it via a driver with AXI slave control and DMA connexion to move data.

We managed to run our full project on the board, calling it from C functions through the driver ; our design achieved a frequency of 72.8MHz.

Table of contents :

- I. Introduction
    1. The CAST-256 algorithm
    2. GCM : authenticating ciphertexts
    3. FPGAs, AXI and DMA
- II. Project overview
- III. Implementation
    1. Driver
    2. DMA engine
    3. AXI slave
    4. CAST-256 and GCM (crypto core)
- IV. Integration
    1. VHDL workflow and structure
    2. Tests and usage
- V. Results and metrics
- VI. Potential upgrades
- VII. AI disclosure

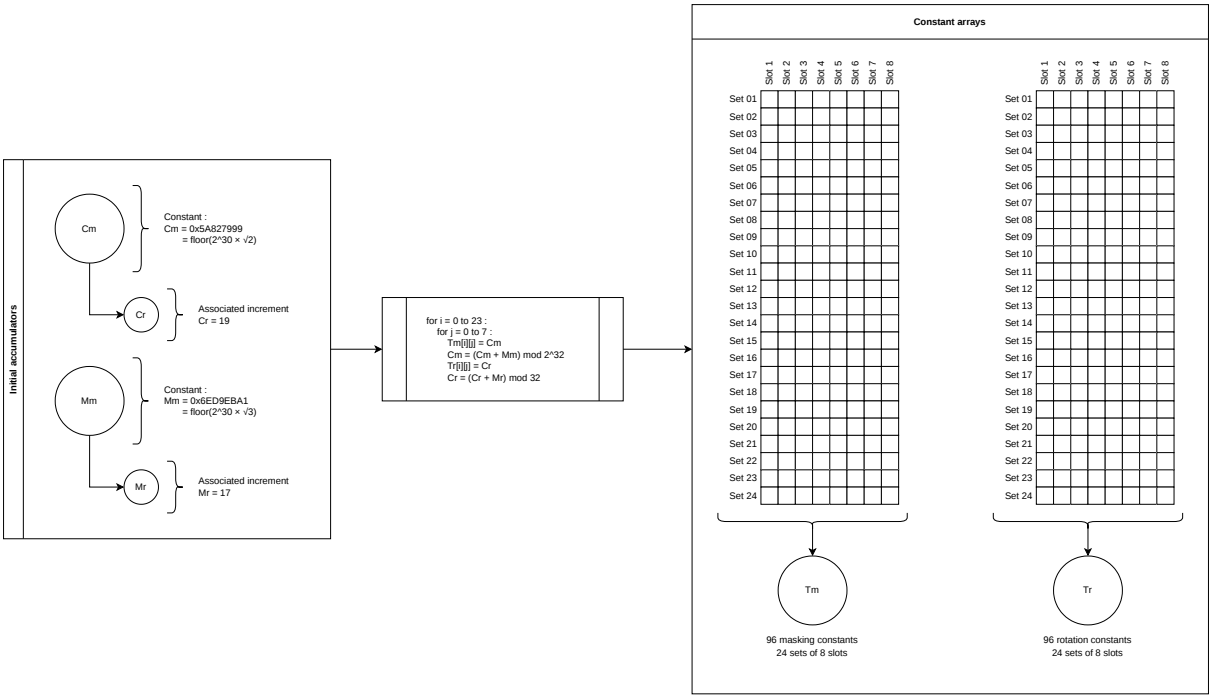
I. Introduction

I.1. The CAST-256 algorithm

CAST-256 is a symmetric block cipher designed by Carlisle Adams and Staffan Tavares at Entrust Technologies in 1998. It was originally a candidate for the AES competition, though it didn’t win : its safety wasn’t questioned, but its execution speed was. This is mainly due to its use of a 48-rounds Feistel network ; but CAST-256 still presents some interesting ideas in order to make it a robust ciphering algorithm, which we tackled the implementation of on the Zybo Z7 SoC.

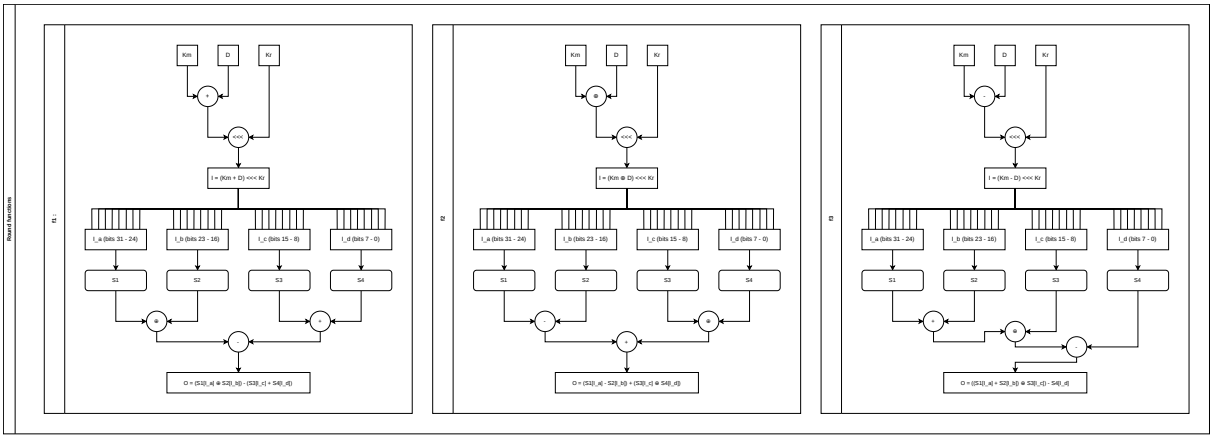
Here is an explanation of how the algorithm works :

- First, we initialize constants of masking and rotation, which will be used later. They are derived from initialization numbers, which are taken from powers of 2 and some prime numbers, rather than picked : this is to prove that the creators did not hide any cryptographic backdoor in their choice of these initial constants.

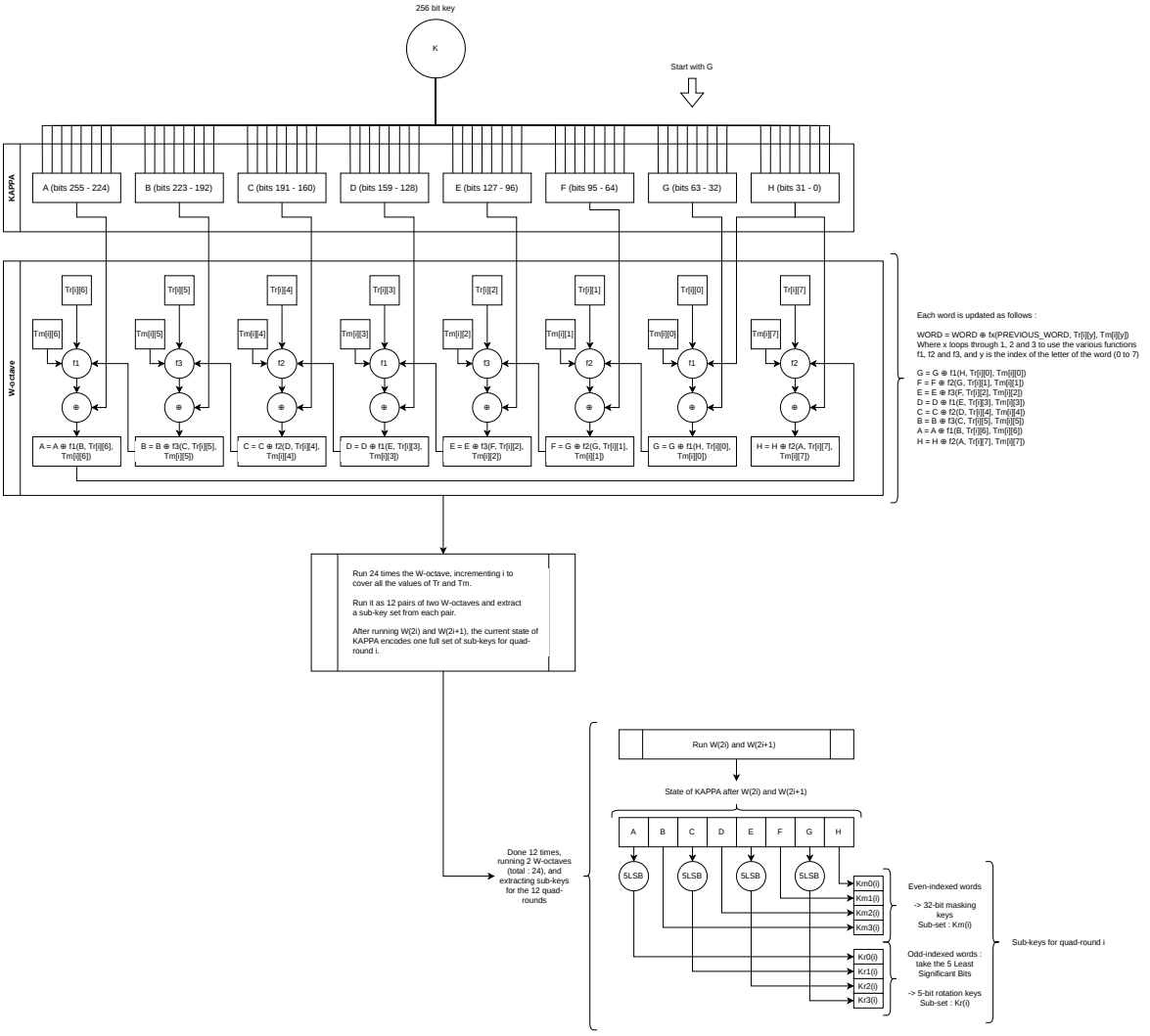


- Then, we define f-functions : there are three of them, that use slightly different operations (additions, XORing, subtraction) to combine the

different parts, so that there is no clear mathematical pattern that could be exploited at some point. The principle is the same for the three : mask the input with a masking constant, rotate it with a rotation constant, pass this through the S-Boxes, recombine the result.

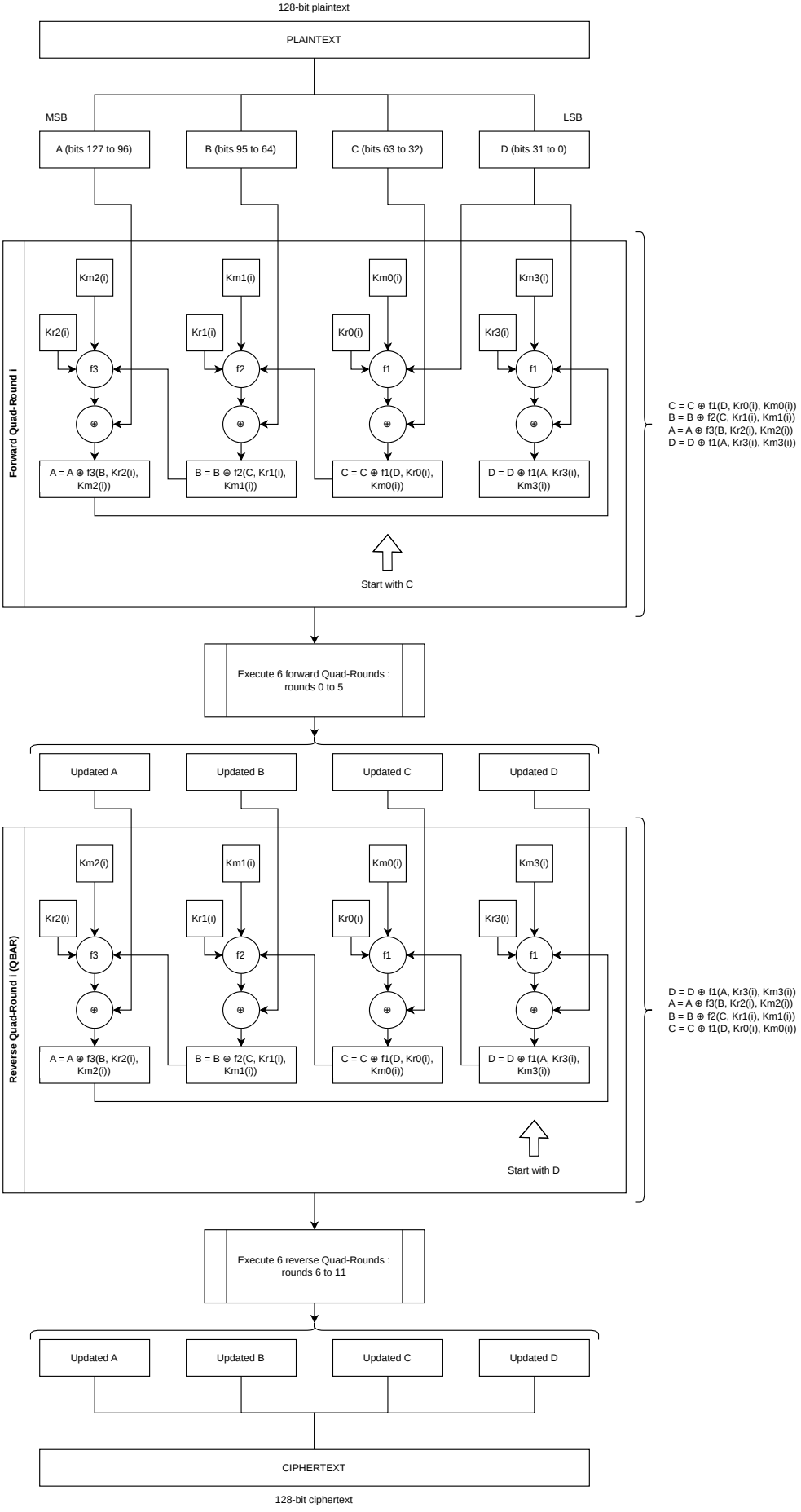


- Then comes the key scheduling, which consists of “W-Octaves” that use these round functions. The goal is to run 24 of them, to extract 12 sets of subkeys (one set every 2 W-Octave)



For each 8-subkey set, 4 of them are 32-bit masking keys, and 4 are 5-bit rotation keys, for a total of 96 subkeys that will all be used in the encryption.

- Final step, the encryption : we execute Quad-rounds, which resemble the W-Octaves, but use the sub-keys this time. 6 of them are regular Quad-rounds, followed by 6 reverse ones (they aren’t linear and do not cancel each other out) ; the idea here is to, again, break any mathematical pattern that may arise.


128-bit plaintext
PLAINTEXT
MSB
LSB
A (bits 127 to 96)
B (bits 95 to 64)
C (bits 63 to 32)
D (bits 31 to 0)
Forward Quad-Round i
Km2(i)
Km1(i)
Km0(i)
Km3(i)
Kr2(i)
Kr1(i)
Kr0(i)
Kr3(i)
f3
f2
f1
f1
⊕
⊕
⊕
⊕
A = A ⊕ f3(B, Kr2(i), Km2(i))
B = B ⊕ f2(C, Kr1(i), Km1(i))
C = C ⊕ f1(D, Kr0(i), Km0(i))
D = D ⊕ f1(A, Kr3(i), Km3(i))
C = C ⊕ f1(D, Kr0(i), Km0(i))
B = B ⊕ f2(C, Kr1(i), Km1(i))
A = A ⊕ f3(B, Kr2(i), Km2(i))
D = D ⊕ f1(A, Kr3(i), Km3(i))
Start with C
Execute 6 forward Quad-Rounds : rounds 0 to 5
Updated A
Updated B
Updated C
Updated D
Reverse Quad-Round i (QBAR)
Km2(i)
Km1(i)
Km0(i)
Km3(i)
Kr2(i)
Kr1(i)
Kr0(i)
Kr3(i)
f3
f2
f1
f1
⊕
⊕
⊕
⊕
A = A ⊕ f3(B, Kr2(i), Km2(i))
B = B ⊕ f2(C, Kr1(i), Km1(i))
C = C ⊕ f1(D, Kr0(i), Km0(i))
D = D ⊕ f1(A, Kr3(i), Km3(i))
D = D ⊕ f1(A, Kr3(i), Km3(i))
A = A ⊕ f3(B, Kr2(i), Km2(i))
B = B ⊕ f2(C, Kr1(i), Km1(i))
C = C ⊕ f1(D, Kr0(i), Km0(i))
Start with D
Execute 6 reverse Quad-Rounds : rounds 6 to 11
Updated A
Updated B
Updated C
Updated D
CIPHERTEXT
128-bit ciphertext

For decryption, the process is exactly the same, but in reverse.

I.2. GCM : authenticating ciphertexts

GCM is an Authenticated Encryption with Associated Data (AEAD) mode of operation, used for block ciphers. It was designed by David McGrew and John Viega at Cisco in 2004, and standardized by the NIST in 2007 as it grew into one of the most widely deployed encryption modes in the world.

By computing a tag during encryption, it allows the users to ensure confidentiality, integrity and authenticity of the data, and gives them the possibility to check whether or not a ciphertext has been tampered with.

I.3. FPGAs, AXI and DMA

FPGAs (Field-Programmable Gate Array) are a very interesting category of chips, which consist of generic logic blocks, hardware that isn't configured to do anything by default. Through Hardware Description Languages (HDLs) such as VHDL, we can program and configure the hardware to execute one task, extremely fast, since all operations are conducted in parallel rather than sequentially.

To target the FPGA of the Zybo board, we use an AXI slave interface to expose the registers to the regular CPU, and allow it to write into them or read from them to control the behavior of the FPGA.

The DMA takes the relay when it comes to moving actual data (like the plaintext) and not just control instructions.

Finally, a driver allows to bridge the gap between the hardware and user software, exposing this to C function calls.

II. Project overview

Now that the different parts of this project have been introduced, we would like to give a global overview of what they do and how they sit together, in a more

detailed manner.

It all starts from the driver, since it is the most top-level exposition of the entire project, making it callable from C functions :

- On `init`, opens `/dev/mem` and mmaps the AXI slave's physical address range into the process's virtual address space ; after that, register reads and writes become just memory accesses
- It uses the Control and Status Register (CSR) to control operations and read back their outcomes
- Provides a clean C API : `crypto_set_key()`, `crypto_set_iv()`, `crypto_encrypt()`, `crypto_decrypt()`, `crypto_get_tag()` (more on each later)
- Each operation will write the job parameters into the AXI slave registers, writes `CSR_START`, then busy-polls `CSR_DONE` (accounting for the time it takes to drain the DMA write buffer)
- Also handles the mock/test build path : the driver code can run on a host machine against a software model of the actual hardware, which is used for unit testing without having to use the board

The next interaction would be the AXI slave interface, responsible for the hardware configuration interface. It's a VHDL block that sits inside the FPGA and lets the CPU (ARM Cortex-A9 on the
Zybo) read/write registers on the crypto accelerator.

- The CPU sends "write register X with value Y" over the AXI bus
- The AXI slave receives it, decodes which register is targeted, and stores the value
- It also exposes registers : `REG_KEY`, `REG_IV`, `REG_AAD_ADDR`, `REG_CSR` (control/status), etc.
- It is described as "slave" because it only reacts to the CPU, which is the master that initiates the operations

Once the CPU has configured the job via the AXI slave, the DMA engine takes the relay to actually transfer the data between DDR memory and the crypto core, without the CPU having to transfer every byte, which would be extremely slow.

- `dma_read_engine.vhd` : fetches plaintext/AAD from DDR → feeds it to the crypto pipeline

- `dma_write_engine.vhd` : takes ciphertext out of the crypto pipeline → writes it back to DDR
- Uses the physical addresses (`REG_PT_ADDR`, `REG_CT_ADDR`) that the CPU wrote into the registers with CSR
- Has a FIFO (`fifo.vhd`) to buffer the data between memory bus bursts and the crypto datapath

The crypto core itself is responsible for key scheduling, encryption, decryption, and computing/checking the tag through GCM.

Here is a simple diagram to understand the top-level interaction of all these components :

```
Linux user-space
        |
        |  crypto_encrypt(aad_phys, pt_phys, ct_phys, ...)
        |
        v
    Driver          <- mmap /dev/mem ->    AXI Slave (FPGA)
 (writes key, IV,                          stores registers
 addresses, START)                         pulses start_i
                                                |
                                           GCM engine (FPGA)
                                           runs CAST-256 + GHASH
                                           block by block
                                                |
                                                | rd/wr requests
                                                |
                                           DMA engine (FPGA)
                                           fetches PT from DDR,
                                           writes CT back to DDR
                                                |
                                           GCM asserts done_o
                                                |
                                           AXI Slave sets CSR_DONE
                                                |
    Driver reads CSR_DONE  <--------------------┘
    returns to user-space
```

We will now move on to the specific implementation of each of these modules, and especially, the design choices that were made hardware-wise.

III. Implementation

Note : in the various directories of the repository are different `README.md` files in order to remind the reader of the usage and structure of each part, for easier navigation and documentation while exploring the code.

III.1. The driver

The driver implementation is at : `/driver`

III.1.A. Description of the different files

- `crypto_driver.h` : public API header that declares all the functions exposed to application code : `init` / `cleanup` / `reset`, key/IV loading, encrypt/decrypt, result readback,
  memory utilities, and the test-build hooks (`-DCRYPTO_TEST_BUILD`).
- `crypto_driver.c` : the core driver implementation. It owns `g_regs`, the `init` / `cleanup` / `reset` lifecycle, `crypto_set_key` / `crypto_set_iv`, `crypto_encrypt` / `crypto_decrypt`
  (including the `wait_done` polling loop), and the readback helpers. In test-build mode, `/dev/mem` is replaced by mock stubs and `wait_done` calls a configurable hook instead of polling real hardware.
- `crypto_mem.c` : Linux/ARM-specific memory utilities : `virt_to_phys` (physical address
  discovery via `/proc/self/pagemap`), `crypto_flush_cache`, and `crypto_invalidate_cache`. Cache operations use the `__ARM_NR_cacheflush` syscall directly ; `__builtin___clear_cache` is avoided because its `dcache` side-effect is toolchain-dependent. Not linked for host-side builds.
- `crypto_regs.h` : the hardware register map of byte offsets, CSR bit masks, `CRYPTO_BASE_PHYS` (`0x43C00000`), map size (4kB), and polling timeout (30s).
- `crypto_errors.h` : nine error codes returned as negative values by all driver functions.
- `driver/crypto_hw_priv.h` : private shim exposing `g_regs` and the `reg_read` / `reg_write` inline helpers (byte offset / 4 indexing).
- `driver/crypto_test.c` : on-board hardware test suite (`UNIT1` - `UNIT5`) for the Zybo with a
  programmed bitstream. Includes diagnostic helpers (`dump_hw_regs` ,

`read_phys_uncached`, `verify_phys_mapping`) and allocates DMA buffers via the double-mapping strategy described later

- `driver/crypto_demo.c` : end-to-end demonstration, which encrypts a 32-byte message with 16-byte AAD, prints ciphertext and tag, decrypts and verifies the plaintext, and then confirms a tampered ciphertext is rejected.
- `driver/tests/test_logic.c` : host-side unit tests (20 groups) covering error paths, input validation, register encoding, and result readback. Uses `crypto_init_mock` ; no ARM
  or `/dev/mem` required.
- `driver/tests/test_e2e.c` : host-side end-to-end tests (`T1` - `T6`) calling the public API as real
  application code would. Results are compared byte-for-byte against an independent C GCM reference (`gcm_ref.c`).
- `driver/tests/hw_model.c` / `hw_model.h` : software model of the full hardware stack. Installed via `crypto_set_hw_hook` : on `CSR_START`, reads register values, resolves physical addresses through a mapping table, runs the C GCM reference, and writes CT, tag, and CSR status back into the mock register array.

III.1.B. Design choices and motivations

III.1.B.1. `/dev/mem` with `O_SYNC` : direct register access, no kernel module

The driver opens `/dev/mem` with `O_SYNC` and `mmap`s the peripheral with `MAP_SHARED`. `O_SYNC` forces the kernel to create a device-type (strongly-ordered, non-cacheable) PTE : every access through `g_regs` goes directly to the AXI bus, which is required for MMIO : a cached read could
in fact return a stale value. Because the hardware signals completion via a CSR bit rather than an interrupt, a polling driver is sufficient, thus avoiding the complexity of a kernel module. The `reg_read` / `reg_write` helpers in `crypto_hw_priv.h` translate byte offsets to word indices (offset / 4).

III.1.B.2. Register map and CSR design

The hardware exposes a flat 4 kB register bank at `0x43C00000` (Zynq-7000 `M_AXI_GP0` window). The driver writes job parameters into registers, asserts `START`, then polls `REG_CSR` for completion.

Offset	Name	Dir	Description
0x000 - 0x01C	REG_KEY	W	Eight 32-bit words, 256-bit key
0x020 - 0x028	REG_IV	W	Three 32-bit words, 96-bit IV
0x030	REG_AAD_ADDR	W	Physical DDR address of AAD buffer
0x034	REG_AAD_LEN	W	AAD byte length (multiple of 16)
0x038	REG_PT_ADDR	W	Plaintext (enc) / ciphertext (dec) address
0x03C	REG_PT_LEN	W	Corresponding byte length
0x040	REG_CT_ADDR	W	Ciphertext (enc) / plaintext (dec) address
0x044	REG_CT_LEN	R	Written by hardware after encrypt
0x048 - 0x054	REG_TAG	R/W	Four 32-bit words, 128-bit auth tag
0x058	REG_CSR	R/W	Control/Status Register
0x05C	REG_DEBUG	R	GCM FSM state in bits [3:0]

Bit	Name	Dir	Meaning
0	START	W	Write 1 to launch an operation
1	SOFT_RESET	W	Write 1 to reset the FSM and clear registers
2	MODE	R/W	0 = encrypt, 1 = decrypt
8	BUSY	R	Operation in progress
9	DONE	R	Operation complete
10	TAG_VALID	R	Authentication tag matched on decrypt
11	ERROR	R	AXI bus error (BRESP or RRESP != 00)

The driver always guards against writing `START` when `BUSY` is set, returning `-CRYPTO_ERR_BUSY` immediately rather than corrupting an in-flight operation.

III.1.B.3. DMA buffer management : the double-mapping strategy

The DMA engine accesses DDR by physical address, so the driver needs a stable physical address for each buffer and a way to access the same memory

from the CPU without stale cache data. The solution used in `crypto_test.c` and `crypto_demo.c` is a double-mapping :

- Step 1 : anonymous `mmap` + `mlock`.
  A `MAP_PRIVATE | MAP_ANONYMOUS` page is touched so the kernel assigns a physical frame, then pinned with `mlock`. `virt_to_phys` reads `/proc/self/pagemap` (PFN in bits [54:0], present flag in bit 63) to obtain the physical address.
- Step 2 : `/dev/mem` remapping with `O_SYNC`.
  The same physical address is mapped again via `/dev/mem` with `O_SYNC` and `MAP_SHARED`. This strongly-ordered mapping bypasses L1/L2 ; all the CPU accesses go directly to DDR, eliminating the need for explicit cache maintenance around every DMA operation.

When a cached allocation is unavoidable, `crypto_flush_cache` must be called before a DMA read (so the DMA sees CPU writes), and `crypto_invalidate_cache` after a DMA write (so the CPU reads fresh data). Skipping either causes silent data corruption.

III.1.B.4. `wait_done` : polling, write-buffer drain problem

After asserting `CSR_START`, the driver polls `REG_CSR` every 10µs with a 30s absolute deadline (`clock_gettime` / `CLOCK_MONOTONIC`). A correctness issue arises at encrypt completion : the DMA write engine asserts `BVALID`, which propagates to `DONE`, from its internal write buffer and not from the DDR controller. The last ciphertext bytes may still be in transit inside `S_AXI_GP0` when `DONE` becomes visible to the CPU. This is why `wait_done` performs two steps on `DONE` ==1:

- A DSB (Data Synchronisation Barrier) to order all prior CPU accesses.
- A 5ms `nanosleep` to let the `S_AXI_GP0` write buffer drain into DDR before the caller invalidates its cache and reads the output.

III.1.B.5. Decrypt tag pre-loading

The expected tag must be written into `REG_TAG` (`0x048` - `0x054`) before `CSR_START` is asserted ; writing it after `START` would create a race where the hardware reads a partially-written tag. `crypto_decrypt` therefore calls `write_tag(expected_tag)` before any other register write. After `wait_done`, it checks for `CSR_TAG_VALID` : if clear, it returns `-CRYPTO_ERR_TAG_MISMATCH`, distinct from `-CRYPTO_ERR_AXI` and `-CRYPTO_ERR_TIMEOUT`.

Note : the address convention for decrypt is the inverse of encrypt, `REG_PT_ADDR` / `REG_PT_LEN` hold the ciphertext input and `REG_CT_ADDR` holds the plaintext output.

III.1.C. Validation

III.1.C.1. Host-side logic tests (`driver/tests/test_logic.c`)

We can build and run the validation with : `make -C driver/tests unit`
It uses `crypto_init_mock` ; no ARM or hardware required.

Test	What is checked
NOT_INIT checks	All API functions return -CRYPTO_ERR_NOT_INIT before crypto_init
Key length validation	Only 16, 24, 32 accepted; all others → -CRYPTO_ERR_INVALID_KEY
BUSY rejection	All config/op functions return -CRYPTO_ERR_BUSY when CSR_BUSY set
Key big-endian encoding	Word layout verified for 16-, 24-, and 32-byte keys; padding checked
IV big-endian encoding	Three words verified at REG_IV_BASE+0/+4/+8
Tag round-trip	crypto_get_tag returns the 16 bytes set in mock registers
Encrypt register programming	REG_AAD_ADDR/LEN, REG_PT_ADDR/LEN, REG_CT_ADDR hold caller values
Encrypt CSR value	mock_csr_at_start == CSR_START, MODE bit clear
Decrypt CSR value	mock_csr_at_start == CSR_MODE
Decrypt tag pre-load	TAG registers hold expected_tag bytes before START
TAG_VALID / TAG_MISMATCH	CSR_DONE
AXI error propagation	CSR_ERROR → -CRYPTO_ERR_AXI for both encrypt and decrypt
Timeout	mock_hw_response=0 → -CRYPTO_ERR_TIMEOUT
NULL pointer guards	All pointer args rejected with -CRYPTO_ERR_INVALID_PARAM when NULL
Zero-length encrypt	aad_len=0, pt_len=0 returns 0; all 5 registers still written
get_ct_len readback	REG_CT_LEN=128 in mock → get_ct_len returns 128
get_status readback	CSR_DONE
Soft reset	crypto_reset writes CSR_SOFT_RESET to REG_CSR

III.1.C.2. Host-side end-to-end tests (`driver/tests/test_e2e.c`)

Build and run : `make -C driver/tests e2e`
`hw_model.c` is installed as a hook so `wait_done` runs the full C GCM implementation. Results are compared byte-for-byte against `gcm_ref.c`.

Test	Description	What is verified
T1	Encrypt (32 B PT, 16 B AAD)	CT and TAG match gcm_encrypt C reference
T2	Decrypt valid tag	Recovered PT matches original; no error returned
T3	Decrypt tampered CT	One flipped byte → -CRYPTO_ERR_TAG_MISMATCH
T4	Encrypt, zero AAD	CT and TAG match reference with NULL AAD
T5	Encrypt, zero PT (auth-only)	TAG matches reference with NULL PT
T6	Two sequential encrypts, diff IVs	Both CTs match references; CTs are distinct

III.1.C.3. On-board hardware tests (`driver/crypto_test.c`)

Build: `make -C driver tests_arm`
Run on Zybo as root : `./crypto_tests_arm`

Test	Description	What is verified
UNIT1	Key + IV load	CSR shows BUSY=0, DONE=0 after config writes
UNIT2	Encrypt to address 0x0	Returns error or timeout; crypto_reset recovers FSM
UNIT3	Encrypt 16 zero bytes	virt_to_phys and cache flush validated via /dev/mem; tag captured
UNIT4	Decrypt with correct tag	Returns 0, CSR_TAG_VALID set, PT reads back as zero
UNIT5	Decrypt with tampered CT	Returns -CRYPTO_ERR_TAG_MISMATCH

III.2. The DMA engine

The DMA implementation is at : `vhdl/dma/`

III.2.A. Description of the different files

- `dma_engine.vhd` : top-level wrapper ; instantiates the read and write engines in parallel and combines their individual error flags into the single error output exposed to the GCM FSM.
- `dma_read_engine.vhd` : 3-state register FSM that drives the AXI4-Lite read address and data channels. Sends raw 32-bit words to `block_join`. Produces sticky completion and error outputs.
- `dma_write_engine.vhd` : 6-state register FSM that drives the AXI4-Lite write address, data and response channels. Consumes 32-bit words from `block_split`. Produces sticky completion and error outputs.
- `block_join.vhd` : assembles four consecutive 32-bit AXI words (big-endian) into a 128-bit block and pushes completed blocks into an internal depth-4 FIFO. Stalls the read data channel when the FIFO is full. The GCM FSM drains the FIFO one block at a time.
- `block_split.vhd` : accepts 128-bit blocks from the GCM FSM into a depth-4 FIFO and streams them out as four sequential 32-bit words to the write engine.
- `fifo.vhd` : course-provided parameterized synchronous FIFO, instantiated at width 128 bits, depth 4, by both `block_join` and `block_split`.
- `tb_dma_engine.vhd` : 8-test self-checking testbench. Uses the course `axi_memory` behavioural model and a `soft_fifo_pkg` reference checker. Every blocking wait carries a 500-cycle timeout that causes the simulation to fail with a named message rather than hang.

III.2.B. Design choices and motivations

III.2.B.1. Parallel read and write engines

During the encrypt phase, the GCM FSM issues `rd_start` and `wr_start` simultaneously : it fetches the next plaintext block while storing the previous ciphertext block. A single serialized engine would force these phases to alternate, which would mean roughly halving the throughput. Splitting it into `dma_read_engine` (owning the AR/R channels) and `dma_write_engine` (owning the AW/W/B channels) as independent VHDL entities lets both channel groups operate concurrently, with no shared state, except `clock` and `reset`.

III.2.B.2. Having `block_join` and `block_split` as standalone components

The Zynq-7010 `HP0` port is 32 bits wide, so each 128-bit cipher block requires exactly four AXI
transactions. The initial design was embedding the word-to-block assembly and disassembly logic directly inside the FSM processes, which created complex nested conditions. Extracting this into `block_join` (4*32 bits = 128 bits) and `block_split` (128 bits = 4*32 bits), each backed by a depth-4 FIFO, separated the data-width conversion from the AXI handshake state machine.

The depth-4 FIFO absorbs rate mismatches between the two sides of each converter. For the read path, `block_join` can only accept a new 32-bit word when it has a free slot, and it signals this via
`join_s_ready`. If the GCM FSM is slow to consume the previous assembled block (e.g. it is mid-
computation), the FIFO fills up, `join_s_ready` is de-asserted, and thus `m_axi_rready` is held low : the DDR controller stalls and waits.
Without the FIFO, the read engine would have to stop issuing AR requests every time the GCM FSM was not immediately ready to receive, which would mean wasting bus bandwidth. Hence why the FIFO decouples the two sides so the read engine can fetch the next words as soon as the bus is free, regardless of when the GCM FSM will consume them.
The same reasoning applies symmetrically to `block_split` on the write path : the GCM FSM can deposit a ciphertext block into the FIFO and immediately move on to the next encryption round, without having to wait for the write engine to complete the four AXI write transactions.

III.2.B.3. 32-bit word assembly : big-endian ordering

Each 128-bit block is assembled MSW-first :

AXI word	Block bits
word 0 (first)	[127:96]
word 1	[95:64]
word 2	[63:32]
word 3 (last)	[31:0]

This matches the network byte order expected by CAST-256 (see RFC 2612, paragraph 2.1) and GCM, eliminating any endian swizzle in the cryptographic datapath.

III.2.B.4. Sticky rd_done / wr_done signals

Both done outputs are level signals, they assert when all blocks have been transferred, and remain
asserted until the next `rd_start` / `wr_start` clears them. But a one-shot pulse lasting exactly one clock cycle can be missed if the GCM FSM samples it on the following edge, whereas a sticky level is readable across any number of cycles, which is why it was chosen.

III.2.B.5. AXI write handshake correctness : `WR_ISSUE_AW_W`

The write engine issues `AWVALID` and `WVALID` simultaneously and must detect when the slave has accepted each channel independently. The tricky case is a slave that holds `AWREADY` or `WREADY` permanently high : on the very first cycle of `WR_ISSUE_AW_W`, the `VALID` signals have not yet been driven high by the registered outputs (and they are still 0 from the previous state). A naive implementation that checks if `awready` == 1 as a sufficient condition for acceptance would be problematic, as it would see `AWREADY` = 1, `AWVALID` = 0, and incorrectly conclude from this situation that the address handshake is complete, when actually the AXI4 handshake requires both
`VALID` and `READY` to be high simultaneously.

The `check_aw_w_acks` procedure avoids this, by checking `awvalid_i` AND `awready` (and similarly for W), where `awvalid_i` is the registered copy of `AWVALID`. On the first cycle of `WR_ISSUE_AW_W` , `awvalid_i` is still 0 , so the condition cannot be validated. The `VALID` outputs are driven to 1 combinatorially in the same cycle, and the registered copy thus catches up on the next rising edge, at which point the slave's `READY` is sampled correctly.

III.2.B.6. The `WR_WAIT_SPLIT` state : decoupled FIFO latency

When the B-channel response arrives but `block_split` has not yet produced the next word
(`split_m_valid` = 0), the FSM parks in `WR_WAIT_SPLIT` rather than looping in `WR_WAIT_BRESP`.
This keeps the branch conditions in `WR_WAIT_BRESP` simple and avoids re-checking the FIFO every cycle while we're waiting for it to fill.

III.2.C. Validation

All tests were run with GHDL 5.1.1 using `make run` in `vhdl/crypto/dma/`. The testbench uses the course `axi_memory template`, and the `soft_fifo_pkg` reference

checker. The simulation terminates with : `DMA ENGINE : ALL TESTS PASSED`

Here are the different tests, and what they test the DMA system for ;

ID	Name	What it tests
T1	Single-block read	Correct big-endian assembly in block_join (MSB-first byte ordering), read-to-GCM handshake completion
T2	Multi-block read	Address pointer advances by exactly 16 bytes per block over 4 consecutive fetches
T3	Write + readback	Big-endian write ordering and end-to-end write→read path through the shared memory model
T4	Parallel read/write	No stalling or traffic corruption when both engines run simultaneously (models GCM encrypt phase)
T5	Read-side bus error	Error detected mid-transfer, transfer halted before completion, error flag asserted to FSM, clean idle after reset
T6	Write-side bus error	Block accepted into write FIFO (FSM not stalled), error flag asserted on bus-level failure before write completes
T7	Read throughput	16 blocks : 695 cycles total, 43 cycles/block (~27 MB/s @ 72.8MHz)
T8	Write throughput	261 cycles for 16 blocks ≈ 16.3 cycles/block (~71 MB/s @ 72.8 MHz)

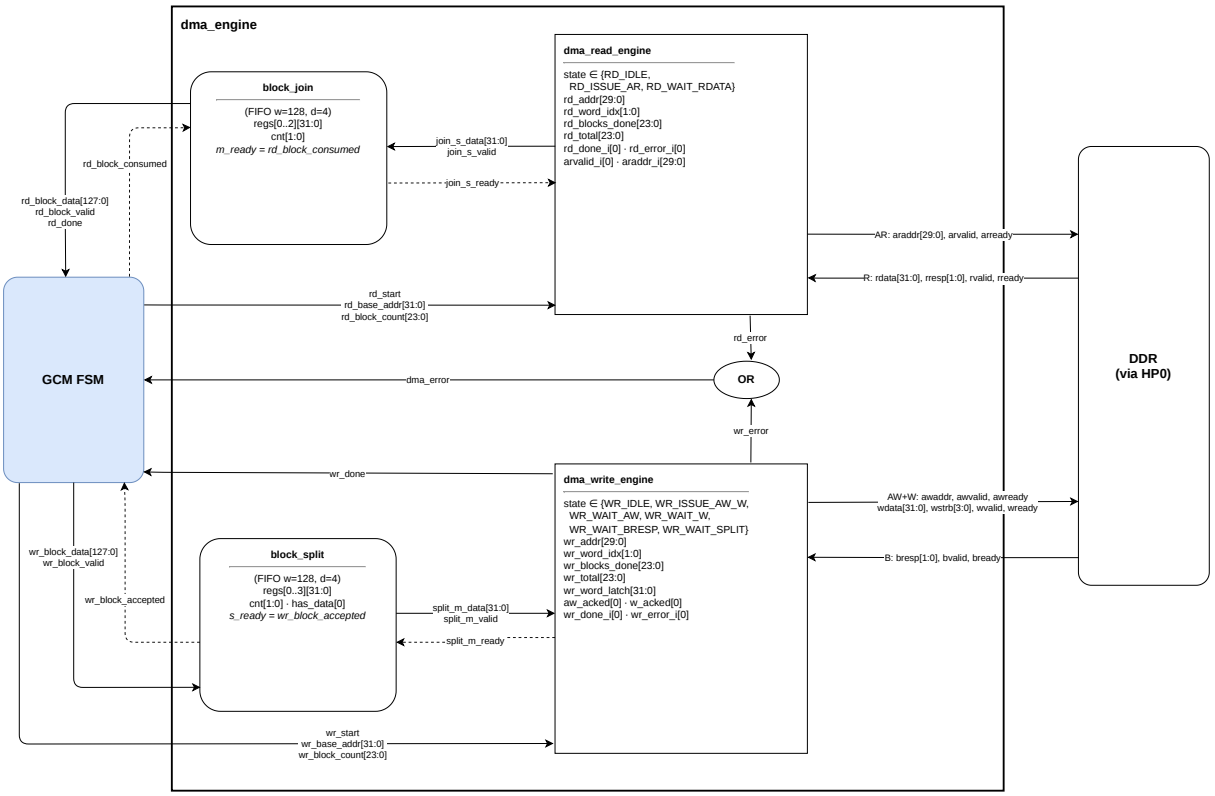
III.2.D. Diagrams

III.2.D.1. Block diagram

The diagram shows the two independent pipelines that share the memory bus.

The read pipeline (top half) fetches data from DDR, assembles it into 128-bit blocks through `block_join`, and delivers them to the GCM FSM.
The write pipeline (bottom half) receives 128-bit ciphertext blocks from the GCM FSM, splits them into 32-bit words through block_split , and writes them to DDR.

The small OR gate at the centre combines the fault signals from both pipelines into a single error output to the GCM FSM.

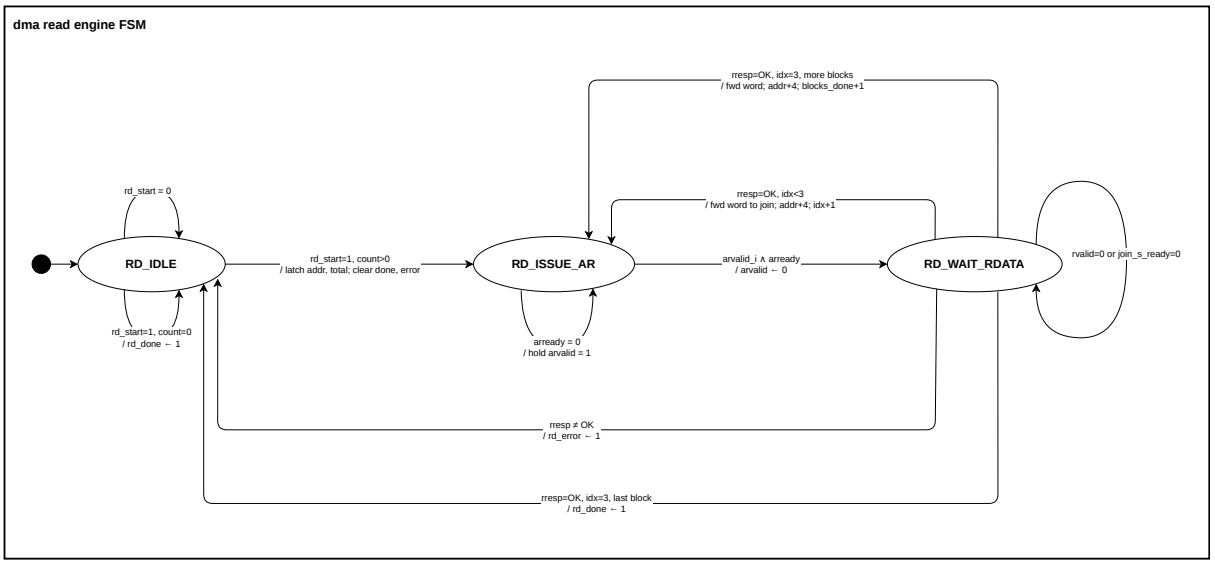


III.2.D.2. Read engine state diagram

The read engine has three states. Reading the diagram is easier with the following description of what each state represents at the logical level.

- `Idle` : the engine is dormant and waiting. It monitors the GCM FSM's start signal ; once it receives a transfer request (a destination address and a block count), it latches those parameters, and immediately moves to the next state. If the requested count is zero, it signals completion and stays idle.
- `Address request` : the engine has placed a read address on the memory bus and is waiting for the memory controller to acknowledge receipt of it. The address is held stable for as many clock cycles as the controller requires before signalling acceptance. The engine cannot advance until that acknowledgement arrives.
- `Data collection` : the engine is waiting for the memory controller to return the 32-bit word it requested. When the word arrives and the assembly buffer has room, the word is forwarded to `block_join` and the engine immediately loops back to request the next address. This repeats four times to collect one full 128-bit block. After the fourth word the assembled block is available to the

GCM FSM ; if more blocks remain the engine issues the next address. Otherwise it signals completion and returns to idle.



Notes :

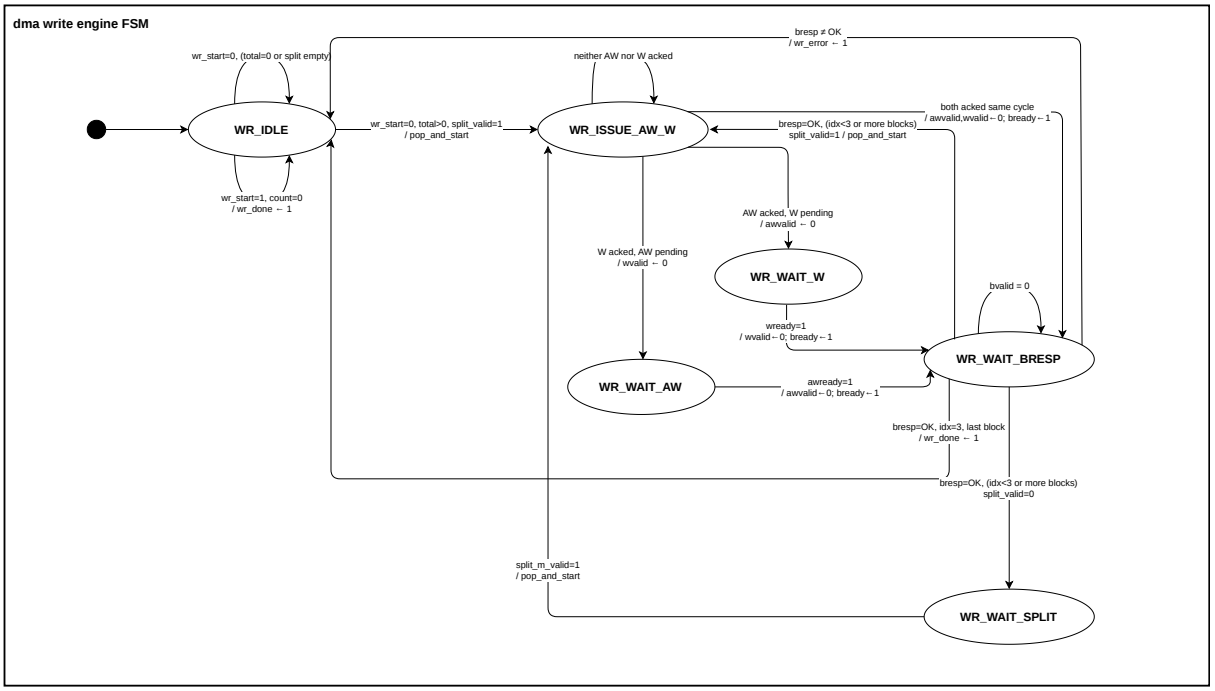
- a bus error that happens, and this at any point, aborts the transfer, raises the fault flag, and returns to idle.
- in the data collection state, the engine's readiness to accept a new word from the bus is gated on the assembly buffer having a free slot. If the buffer is full, the engine stalls the memory bus until the GCM FSM consumes the waiting assembled block.

III.2.D.3. Write engine state diagram

The write engine has six states. The additional complexity compared to the read engine comes from the AXI4-Lite write protocol : a write transaction requires three independent handshakes (address, data, and write response), which can complete in any order, and the engine must track all three simultaneously.

- `Idle` : the engine is dormant. It waits until the GCM FSM arms it with a destination address and block count, and until the first 32-bit word from the splitter is ready to send.

- `Issue address and data` : the engine places the write address and the data word on the bus
  simultaneously and waits for the memory controller to accept both.
  Because the two channels are independent, acceptance may arrive in any order ; the engine handles all three cases : both accepted at once, address first, or data first.
- `Wait for address acknowledgement` : the data word was accepted first but the write address has not yet been acknowledged. The engine holds the address on the bus and waits for the controller to signal acceptance before proceeding.
- `Wait for data acknowledgement` : the write address was accepted first but the data word has not yet been acknowledged. The engine holds the data on the bus and waits.
- `Wait for write response` : both the address and the data have been accepted. The engine waits for the memory controller to confirm that the write was stored without error. On a successful response it moves on to the next 32-bit word (or signals completion if this was the last word of the last block). On a bus error it raises the fault flag and aborts.
- `Wait for splitter` : the write response has arrived and the engine is ready to issue the next 32-bit word, but the splitter component has not yet made it available. The engine parks here until the word is ready, then immediately resumes without any extra latency.
- `pop_and_start` : latch the current word from the splitter ; pulse the splitter's consume signal for one clock cycle ; clear the per-channel acknowledgement flags ; transition to the
  issue state.



III.3. The AXI slave

The DMA implementation is at : `vhdl/axi_slave`

III.3.A. Architecture

First, a description of the channels and signals being used across this AXI slave implementation. The channels are :

- a write address channel (`AW`)
- a write data channel (`W`)
- a write response channel (`B`)
- a read address channel (`AR`)
- a read data channel (`R`)

Each of these channels uses a two-signal handshake : `VALID` (driven by the sender) and `READY` (driven by the receiver). The write FSM manages the `AW` , `W` , and `B` channels.The read FSM manages `AR` and `R` .

The AXI slave implementation is split into four files :

- `axi_slave_pkg.vhd` : holds the shared definitions of the response constants (`RESP_OKAY` , `SLVERR` , `DECERR`) and the FSM state types. It also contains a small

`is_mapped` utility function used to validate addresses.

- `axi_slave_wr.vhd` : the write channel that owns all the registers and drives the `AW`, `W`, and `B` channels. It is the only entity that actually holds state.
- `axi_slave_rd.vhd` : the stateless read channel that receives `tag_in` and `mode_in` directly from the write channel, and uses them to answer `AR` and `R` transactions.
- `axi_slave.vhd` : the wrapper that actually instantiates the two and wires them together correctly. The idea behind keeping the two channels separate is to make the FSMs shorter and easier to validate independently.

III.3.B. Register map

The register map is as follows :

Address	Register	Access	Width
0x000 - 0x01C	Key K (8 × 32-bit words)	Write-only	256 bits
0x020 - 0x028	IV (3 × 32-bit words)	Write-only	96 bits
0x02C	Reserved	-	DECERR
0x030 - 0x034	AAD base address / byte length	Write-only	32 bits each
0x038 - 0x03C	Plaintext base address / byte length	Write-only	32 bits each
0x040	Ciphertext base address	Write-only	32 bits
0x044	Ciphertext byte length	Read-only	32 bits
0x048 - 0x054	Authentication tag (4 × 32-bit words)	R/W	128 bits
0x058	Control and Status Register (CSR)	R/W	see below
0x05C	Debug (FSM state)	Read-only	4 bits

For address registers (`AAD`, `PT`, `CT`), bits 31:30 must be zero as the DMA interface only covers a 30-bit address space. A non-zero value returns a slave error.

Here are a few details for the implementation and design choices :

- The ciphertext byte length at `0x044` is read-only because it is computed by the datapath. In encryption mode it equals the plaintext byte length, so there is no need for the CPU to write it.

- The tag register (`0x048` - `0x054`) is marked `R/W` because its direction depends on the mode. In encryption mode, the datapath writes the computed tag when `csr_done` rises. In decryption mode, the CPU writes the expected tag before asserting `START`, and the accelerator uses it to verify the authentication.
- The CSR (`0x058`) bit fields are detailed below :

Bit	Name	Access	Description
0	START	Write	Launches the accelerator. Ignored if busy
1	SOFT_RESET	Write	Clears all registers. Effective even when busy
2	MODE	R/W	0 = encryption, 1 = decryption
8	BUSY	Read	Accelerator is running
9	DONE	Read	Processing completed
10	TAG_VALID	Read	Authentication tag is valid
11	ERROR	Read	An error occurred during processing

III.3.C. The FSM for writing

The FSM for writing has 4 states (`WR_IDLE`, `WR_HAVE_ADDR`, `WR_HAVE_DATA`, `WR_BOTH`). As the write address (`AW`) and the write data (`W`) are not guaranteed to arrive during the same cycle, we decided to implement intermediate states. In an intermediate state (`WR_HAVE_ADDR` or `WR_HAVE_DATA`), once the missing half arrives, the FSM moves on to `WR_BOTH`.

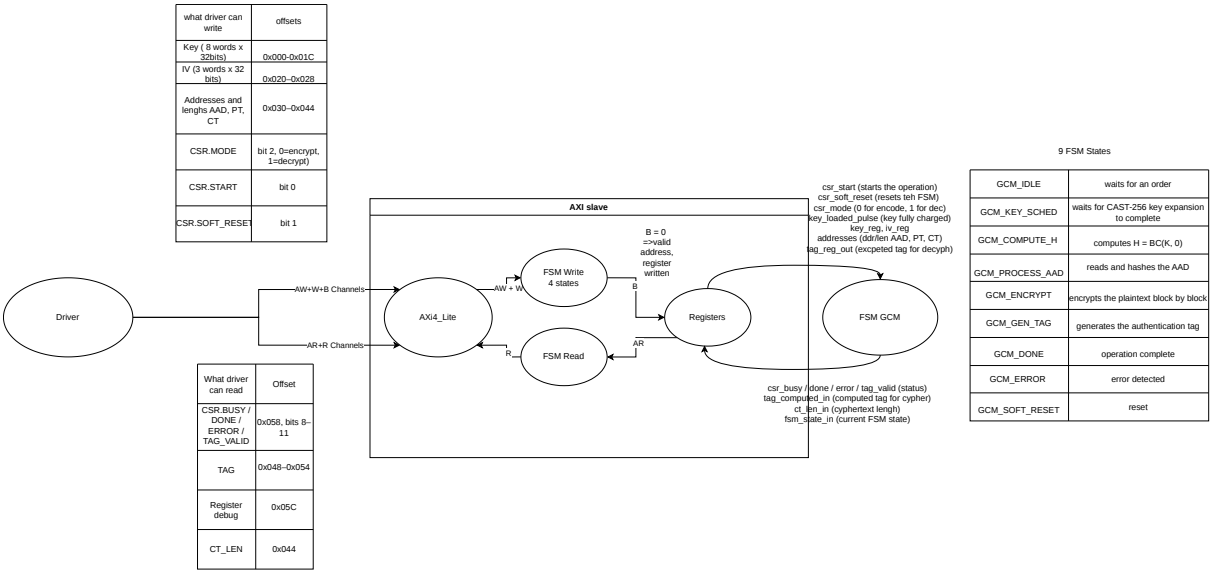
Once in `WR_BOTH`, the address is decoded and `BVALID` is asserted. The register is updated only if the accelerator is not busy (i.e. `csr_busy` is set to 0). If the accelerator is busy, all writes are silently ignored (because the register values cannot change while the accelerator is running). In that case the response is still `OKAY`, as ignoring a write while busy is not considered an error.

`SOFT_RESET` is the only exception : it bypasses the guard and clears all registers regardless. In encryption mode, the tag register is also updated automatically by the datapath when `csr_done` rises, without any CPU write. The FSM then returns to `WR_IDLE` once `BREADY` is seen.

III.3.D. The FSM for reading

The read FSM is simpler, with only 2 states (`RD_IDLE` and `RD_LATCHED`). In the `RD_IDLE` state, the slave is waiting for `ARVALID` . Once the address arrives, it is latched, `ARREADY` is asserted combinatorially, so the handshake completes in the same cycle. The FSM then moves to the `RD_LATCHED` state. The address is decoded, `RDATA` is set, and `RVALID` is asserted. Once `RREADY` is seen, `RVALID` is de-asserted and the FSM returns to `RD_IDLE` .

III.3.E. Diagram



III.4. The crypto core : CAST-256 and GCM

The crypto core is implemented at : `vhdl/crypto`

III.4.A. Description of the different files

- `vhdl/crypto/crypto.vhd` : the top-level integration, which instantiates `axi_slave` , `dma_engine` , and `gcm` . The `p_dma_adapter` process bridges the per-block request model of `gcm` to the burst model of `dma_engine` . Exposes the full AXI4-Lite slave and master port sets to the Zynq PS.
- `vhdl/crypto/gcm/gcm_pkg.vhd` : the shared constants and utilities : the GF(2^{128}) reduction polynomial `GF_R` and the `inc32` function (increment rightmost 32 bits of a 128-bit block, used to generate GCTR counter blocks).

- `vhdl/crypto/gcm/gf128_mul.vhd` : the GF(2^{128}) multiplier (NIST SP 800-38D Algorithm 1). Processes 8 bits of the X operand per clock cycle (16 cycles for 128 bits) then one `DONE_ST` pulse : 17 cycles total latency.
- `vhdl/crypto/gcm/ghash.vhd` : sequential GHASH (SP 800-38D Algorithm 2). Maintains a 128-bit running accumulator `Y`. Each block is `Y = (Y XOR X) · H` via one `gf128_mul` invocation. Latency : 33 cycles per block (1 latch + 17 multiply + pipeline). Resets via `clear_i`.
- `vhdl/crypto/gcm/gcm.vhd` : the GCM top-level FSM (13 states). Orchestrates key schedule, H
  precomputation, AAD hashing, GCTR encrypt/decrypt loop, length-block GHASH, and tag generation/verification. Issues per-block DMA read and write requests. Contains two `cast256` channels and one `ghash` instance.
- `vhdl/crypto/cast256/cast256_pkg.vhd` : holds the S-box constants (SBOX1 - SBOX4), shared array types, and helper functions used by both `cast256_ks` and `cast256_enc`.
- `vhdl/crypto/cast256/cast256_ks.vhd` : the key scheduler, which expands the 256-bit key into 192 round sub-keys over ~396 cycles (24 octaves * 8 f-functions, 2 cycles each, plus 12
  subkey extraction steps). S-boxes use Block RAM (4*RAMB18).
- `vhdl/crypto/cast256/cast256_enc.vhd` : the encryption datapath. 12 rounds (6 forward Q + 6 reverse QBAR), 48 f-function steps, ~50 cycles per 128-bit block. S-boxes use LUTRAM
  (`ram_style="distributed"`) so multiple instances can run in parallel without exhausting BRAM. Includes address-pipelining and forwarding optimisations to halve latency vs the BRAM baseline.
- `vhdl/crypto/cast256/cast256.vhd` : the CAST-256 top-level. One `cast256_ks` instance (BRAM) and two parallel `cast256_enc` instances (LUTRAM), both sharing the subkey buses. The two channels can be active simultaneously ; `encrypt_start_N` is gated on `key_ready` to prevent premature starts.
- `vhdl/crypto/testbenches/tb_crypto_integration.vhd` : functional correctness testbench for `crypto.vhd`. Drives the AXI slave with a behavioural master and answers the DMA master with a memory model. 16 named tests (T0 - T11 + edge cases).
- `vhdl/crypto/testbenches/tb_crypto_bench.vhd` : cycle-accurate performance benchmark. Free-running cycle counter and DMA-monitor process. Sweeps

PT length, AAD length, warm/cold key, encrypt/decrypt across 9 sections (S1 - S9). Prints total cycles, per-phase breakdown, and Mbps throughput.

- `vhdl/crypto/testbenches/tb_cast256.vhd` : smoke test for the `cast256` entity in isolation. Drives one key + plaintext pair and prints the resulting ciphertext for manual verification. The full RFC2612 KAT suite is run separately from `vhdl/crypto/cast256/`.

III.4.B. Design choices and motivations

III.4.B.1. GCM FSM state sequence

The GCM engine is a 13-state Moore FSM. On a cold start (new key), the full sequence is :

```
IDLE → KS_START → KS_WAIT           (run CAST-256 key schedule, ~396 cy)
     → H_START  → H_WAIT            (compute H = CAST256(K, 0¹²⁸), reset GHASH)
     → AAD_CHECK → [AAD_REQ → AAD_WAIT → AAD_HASH] * n   (GHASH each AAD block)
     → GCTR_INIT                    (build J0 = IV‖0³¹‖1, CB₁ = inc32(J0))
     → PT_CHECK → [GCTR_ENC → GCTR_WAIT] * (n-1) → GCTR_LAST
     → LEN_HASH → LEN_WAIT          (GHASH over len(A)‖len(C))
     → TAG_OUT → DONE
```

On a warm start (same key as the previous call), `KS_START` through `H_WAIT` are skipped entirely, which saves ~450 cycles ; the overhead is only present when the key is entirely new, meaning it gets mitigated over many runs and greatly speeds things up on average. The key and H value are cached and reused as long as the same key is presented.

For decryption, src/dst addresses are swapped and GHASH is fed the ciphertext rather than the XOR result. Tag verification happens in `TAG_OUT` : the computed tag is compared against `tag_i` and `auth_fail_o` is asserted on mismatch.

III.4.B.2. GCTR pipeline : CAST-256 and DMA overlap

In the GCTR loop, the GCM FSM issues `CAST256(CB_i)` and a DMA read for `P_i` simultaneously. While CAST-256 runs (~50 cycles), GHASH over `C_{i-1}` runs concurrently. Since the GHASH latency (33 cycles) is below the CAST-256 window (50 cycles), GHASH always completes before the next ciphertext block is ready. The DMA write for `C_i` is issued immediately after the XOR without waiting for `dma_wr_done_i`, except on the final block (`GCTR_LAST`) where the FSM waits to ensure the last write reaches DDR before asserting `done_o`.

Net throughput : 1 block per ~52 cycles (measured), converging towards ~179 Mbps at 72.8 MHz for large warm payloads.

III.4.B.3. Two CAST-256 channels for parallel tag encryption

The `cast256` top-level exposes two independent encryption channels (channel 0 for GCTR counter-block encryption, channel 1 for `CAST256(J0)` used in tag computation).
In `LEN_HASH`, the GCM FSM starts channel 1 on `J0` in parallel with the final GHASH round. Since both take ~50 cycles and GHASH finishes in 33, channel 1 is always ready before `LEN_WAIT` exits. This eliminates an otherwise sequential 50-cycle stall at the end of every operation.

III.4.B.4. LUTRAM S-boxes in `cast256_enc`, BRAM S-boxes in `cast256_ks`

The key schedule (`cast256_ks`) uses BRAM (4 × RAMB18) because it runs once per key and its S-boxes are not on the critical timing path. The encryption datapath (`cast256_enc`) uses LUTRAM (`ram_style="distributed"`) so that two instances can run in parallel without doubling BRAM usage. Each 256*32 S-box maps to ~128 LUT6 cells ; the total resource cost for the two encryption engines is of ~5,400 LUTs with 0 RAMB18.

III.4.B.5. Address-pipelining and forwarding in `cast256_enc`

The original BRAM-based design used an explicit `ADDR` state before each `COMPUTE` state to drive S-box addresses, giving 2 cycles per f-function step and ~97 cycles per block. The optimised design pipelines the address phase : during `COMPUTE` for step N, combinatorial logic simultaneously drives S-box addresses for step N+1 using forwarded intermediate values (`β XOR f_result`). This eliminates the `ADDR` state from the steady-state loop, halving per-step cost and helping reduce the latency to ~50 cycles per block.

Forwarding is only required in the Q rounds (steps 0 - 5) : each step reads the word updated by the previous step, creating a one-cycle data hazard. In QBAR rounds (steps 6 - 11), dependencies are 2+ cycles apart and all words are correctly registered before they are read, therefore no forwarding is needed.

III.4.C. Validation

III.4.C.1. Functional integration tests (`tb_crypto_integration.vhd`)

Build and run with : `make run` from `vhdl/crypto/`
The crypto core is put between two simulation models : a behavioural PS model writes config registers and reads status over the AXI slave port, while a DDR memory model answers the DMA read/write requests issued by the AXI master port. Each test reports PASS/FAIL via `report` statements.

Test	Description	What is verified
T0	Reset state	CSR reads zero after hardware reset
T1	Config write / readback	KEY, IV, addresses written; MODE and CT_LEN readable
T2	Address bounds check	PT_ADDR with bits[31:30] set → SLVERR
T3	Encrypt, zero-length	START → DONE without DMA; tag stable
T4	Decrypt, correct tag	DONE; auth_fail = 0
T5	Decrypt, corrupted tag	DONE; CSR.ERROR = 1
T6	Soft reset	SOFT_RESET clears DONE and TAG_VALID
T7	Encrypt 1-block AAD + 2-block PT	TAG matches reference
T8a	Decrypt round-trip	Recovered PT matches original plaintext
T8b	Tampered CT	CSR.ERROR = 1
T9	Back-to-back encrypt	Key cache reused; DMA adapter re-armed correctly
T10	AAD=0, PT=1 block	GCTR-only DMA path; no AAD phase
T11	4-block PT burst, no AAD	Round-trip encrypt/decrypt passes
T_SWB	START while BUSY	Second START pulse → SLVERR
T_SRESET_MID	Soft reset mid-operation	BUSY clears immediately; FSM returns to IDLE
T_DMA_RD_ERR	Read SLVERR from memory	csr_error asserted

Test	Description	What is verified
T_DMA_WR_ERR	Write SLVERR from memory	csr_error asserted

III.4.C.2. Performance benchmark (`tb_crypto_bench.vhd`)

Build and run with : `make bench` from `vhdl/crypto/`
Cycle-accurate ; it uses a free-running counter and a DMA-monitor process (±1 cycle resolution) for measurement. Memory model latency : 4 cycles (AR → R valid).

Section	Sweep	Metric reported
S1	AXI control-plane	Cycles from write to BVALID
S3	Encrypt, warm key, PT=1..16 blocks	Total cycles, per-phase breakdown, Mbps
S4	Encrypt, cold key, PT=1..8 blocks	Cold vs warm overhead
S6	Decrypt, warm, PT=1..8 blocks	Round-trip cycle count
S8	Per-component cycle estimates	Derived from S3 marginal analysis

Key numbers obtained :

Metric	Value
Key schedule (cold start)	~396 cy
H precomputation	~50 cy
Fixed overhead per operation (floor)	~70 cy (init + tag phase)
Marginal cost per 128-bit block (warm)	52 cy (520 ns)
DMA read bandwidth	24 cy / 128-bit block
DMA write bandwidth	~13 cy / 128-bit block
GHASH per block	33 cy (hidden inside CAST-256 window)
Measured throughput at 8 blocks	~148 Mbps
Asymptotic throughput (warm, large PT)	~179 Mbps

III.4.C.3. CAST-256 KAT tests (`cast256/tb_cast256.vhd`)

The CAST-256 implementation was validated against a reference CAST-256 C code by Brian Gladman, the most cited C standalone reference, included in

`libmcrypt` : https://github.com/winlibs/libmcrypt/blob/master/modules/algorithms/cast-256.c

This reference was used to create KAT files through a Bash script, and then we could validate that our own CAST-256 implementation was correct against these KAT files, testing regular encryptions as well as RFC 2612 Appendix A vectors and edge cases. Additional custom vectors can be supplied via `make run KEY=<64 hex> PT=<32 hex>`. The full 256-bit KAT suite is also exercised via `test_suite/run_kat.sh`.

Build and run with : `make run` from `vhdl/crypto/cast256/`

III.4.D. Latency and throughput summary

Component / phase	Latency	Notes
gf128_mul	17 cy	8 bits/cycle x 16 steps + DONE pulse
ghash per block	33 cy	1 latch + 17 mul + pipeline
cast256_ks (key schedule)	~396 cy	24 octaves, BRAM S-boxes
cast256_enc per block	~50 cy	LUTRAM S-boxes, pipelined addresses
GCM cold start (key miss)	~450 cy	ks + H + GHASH reset
GCM fixed floor per operation	~70 cy	GCTR init + LEN_HASH + TAG_OUT
GCM marginal per block (warm)	52 cy	CAST-256 + DMA read, overlapped
System throughput (8 blocks)	~148 Mbps	cycle-count scaled to 72.8 MHz
System throughput (asymptote)	~179 Mbps	warm key, large PT, 72.8 MHz

IV. Integration

IV.1. VHDL workflow and structure

The build process is driven by four TCL scripts located in `tcl_files/` :

```
vivado -mode batch -source tcl_files/<script>.tcl
```

There are two separate workflows :

- one for fast timing/utilization analysis (OOC synthesis only)
- one for the full FPGA build that produces a bitstream and can be programmed onto the board.

The scripts are :

Script	Purpose
synth_cast256.tcl	OOC synthesis of cast256 alone. Fast feedback on the cipher core timing without synthesizing GCM, AXI slave, or DMA.
synth_crypto.tcl	OOC synthesis of the full crypto fabric (CAST-256 + GCM + AXI slave + DMA). Timing and utilization analysis without Zynq PS.
package_ip.tcl	Packages the crypto VHDL entity as a Vivado IP core (kah:crypto:crypto:1.0). Must run before create_bd.tcl. Output: vhdl/crypto/crypto_ip/.
create_bd.tcl	Creates a Vivado project, builds the full block design (PS + SmartConnect + crypto IP), runs synth → impl → bitstream. Output: vhdl/crypto/fpga.bit and vhdl/crypto/crypto.xsa.

IV.2. Full build and board programming workflow

Here are the different steps to build the project and program the board :

- Step 1 : package the crypto IP (to run once per VHDL change)

  ```
  vivado -mode batch -source tcl_files/package_ip.tcl
  ```

  This reads all the VHDL source files in dependency order, and creates a temporary
  project. Then, it calls `ipx::package_project`. The result is a `component.xml` file in `vhdl/crypto/crypto_ip/` that Vivado recognises as a local IP catalog entry.

  Note : if the folder already exists, the script exits without re-packaging. `vhdl/crypto/crypto_ip/` needs to be deleted to force a rebuild.

- Step 2 : build the block design, synthesize, implement, write the bitstream

```
vivado -mode batch -source tcl_files/create_bd.tcl
```

The script :

- Creates a Vivado project for the `xc7z010clg400-1` (Zybo Z7-10)
- Registers the packaged IP from `vhdl/crypto/crypto_ip/`
- Builds the block design :
  - PS7 `M_AXI_GP0` → SmartConnect → `crypto_0.s0_axi` @ `0x43C00000`
  - `crypto_0.m0_axi` → SmartConnect → PS7 `S_AXI_GP0` (DDR, 512 MB)
  - `FCLK_CLK0` (100 MHz target) drives all fabric clocks
  - `FCLK_RESET0_N` → `proc_sys_reset` → `crypto_0.aresetn`
  - `crypto_0.irq` → `xlconcat` → `PS7.IRQ_F2P[0]`
  - Board ports `sw[3:0]`, `btn[3:0]`, `led[3:0]` wired to crypto
- Generates the HDL wrapper, adds `vhdl/crypto/crypto.xdc` constraints
- Runs synthesis then implementation with `write_bitstream`
- Copies the bitstream to `vhdl/crypto/fpga.bit` and exports `vhdl/crypto/crypto.xsa`

If Digilent board files are not installed, the PS7 is configured with a manual parameter set (DDR3 MT41K256M16, 667 MHz APU, UART0 on MIO 14-15, corresponding to the Zybo Z7).

- Step 3 : program the board

  The board is already configured to accept an `fpga.bit` file as programming of the FPGA at boot. Therefore, we just need to put the file on the SD card for it to be used. Alternatively, one can do the following :

```
Open Vivado Hardware Manager :
Flow Navigator → Open Hardware Manager → Open Target → Auto Connect
→ Program Device → select `vhdl/crypto/fpga.bit` → Program
```

- Step 4 : cross-compile the driver for Cortex-A9

```
cd driver && make tests_arm
# or for the demo binary:
cd driver && make demo_arm
```

Requires `arm-linux-gnueabihf-gcc` in PATH, produces `driver/crypto_tests_arm` or `driver/crypto_demo_arm`.

- Step 5 : deploy and run on the Zybo

  We put the files on the SD card of the board, but alternatively with a connexion set up, one can run :

```
scp driver/crypto_tests_arm root@<zybo-ip>:/root/
ssh root@<zybo-ip> sudo ./crypto_tests_arm
```

IV.3. OOC Synthesis for timing analysis (no board needed)

To check timing and utilization after a VHDL change, without going through the full build (which takes around 15 minutes) :

```
# Full fabric (GCM + CAST-256 + AXI + DMA):
vivado -mode batch -source tcl_files/synth_crypto.tcl

# CAST-256 only (fastest iteration loop):
vivado -mode batch -source tcl_files/synth_cast256.tcl
```

Both scripts run out-of-context synthesis (`-mode out_of_context`), meaning the Zynq PS and DDR/FIXED_IO ports are excluded. This makes synthesis much faster (less than a minute) than a full project build but does not produce a bitstream.

Reports are written to :

Script	Output directory	Reports
synth_crypto.tcl	vhdl/crypto/synth/	timing_summary.rpt
		utilization.rpt

Script	Output directory	Reports
		utilization_hier.rpt
synth_cast256.tcl	vhdl/crypto/cast256/	timing_summary.rpt
		utilization.rpt

The scripts also print a summary to the terminal: WNS, critical path (ns), estimated Fmax, and LUT/FF/DSP/BRAM utilization percentages.

V. Results and metrics

We achieved a critical path of 13.7ns with a maximum clock frequency of 72.8MHz.
Here are some other metrics extracted from the test benches and the synthesis of Vivado :

- Timing & Utilization (Vivado synthesis) :

Metric	Value
Critical path	13.745 ns
Estimated Fmax	72.8 MHz
WNS	-3.745ns @ 100MHz
Slice LUTs	11,587 / 17,600 (65.8%)
Slice Registers	7,742 / 35,200 (22.0%)
RAMB18	4 / 120 (3.3%)
DSPs	0 / 80 (0%)

- Instance cell breakdown :

Module	Cells
Total crypto	21,290
gcm	18,297
cast256 (total)	13,495
cast256_ks	6,214
cast256_enc[0]	3,856

Module	Cells
cast256_enc[1]	3,390
ghash + gf128_mul	1,867
dma_engine	1,049
axi_slave	1,254

- AXI control-plane latency (S1) :

Operation	Cycles	Per access
Write x8 (KEY)	31 cy	~4 cy/write
Read x8 (CSR)	32 cy	4 cy/read

- Key schedule overhead (S2) :

Cold → warm saving : 451 cy = ~6.2 us at 72.8 MHz.

Mode	Cycles	Notes
Cold start	515 cy	Full key schedule + H computation
Warm (hit)	64 cy	Cache hit, skips ks and H entirely

- Encrypt throughput, warm key, no AAD (S3) :

Cycle counts from simulation ; ran at 100MHz and results were rescaled to 72.8 MHz (∗0.728). Marginal cost is stable at 52 cy/block from PT=2 onwards. Fixed floor per operation (warm): ~80 cy (init + tag phase).

PT blocks	Total cy	Marginal cy/block	Mbps @ 72.8 MHz
1	137	-	68
2	189	52	98
3	241	52	116
4	293	52	127
5	345	52	135
6	397	52	141
7	449	52	145
8	501	52	148
inf (asymp)	-	52	~179

VI. Potential upgrades

We see some upgrades that we could have implemented, but have run out of time for. For example, we do talk about saving BRAM and instead using LUTs at some point ; but we do see that we only use 3.3% of the BRAM, but a high percentage of the LUTs. We could go for a more BRAM intensive approach since we can afford it, and free LUTs to implement more crypto engines.

Also, some pipelining could be performed in the `cast256_enc` adder to increase the throughput : the bottleneck clearly lies there, and keeps us at 72.8MHz.

The GF(2^128) multiplier could also be made faster by switching it to 16 bits instead of the current 8.

VII. AI disclosure

Concerning our use of AI, we want to disclose that :

- some files have entirely been written by AI ; those are the ones that were less interesting and not useful for us to code ourselves, for example, the script to extract KAT files of the reference CAST-256 implementation, some testing files that validate behavios, or some Makefiles. We decided to generate them because they are not critical logic, to speed up our process
- some files have been written partially by AI : we have leveraged it to go faster, creating bases on which we could work for some VHDL, or starting our test benches so that we could then adapt them, etc.
- some technical parts of this report where written with AI, as it is good at summarizing complex codebases into a clear and readable way. We believe that what it produced is more readable than what we could have provided.

But we do wish to insist on the fact that any and all AI use has been made under strict human supervision of the results. The reflexions were read, the scripts scrutinized and validated, and the texts humanified : the AI was a tool, and our prompts were not “implement the project”.

The AI has been a great tool to work with, especially its agentic mode, as our codebase is extremely large. Having an agentic instance was new for us, but it was interesting as it allowed us to pull information out of any file, at any time, and easily keep the model up to date with the context and the purpose of our task without having to reupload all the files, all the time.

We noticed that AIs are not yet extremely comfortable with VHDL logic. They succeed greatly at the syntax, which makes sense ; but when we reach the actual logic of designing a system, or improving performances, they often fail to provide useful ideas, and often miss critical elements and propose changes that would completely break the entire system, or greatly diminish its capabilities.

| CANO Louis | KAH Frédéric | HIRSCHMILLER Thomas |